

Multi-Tenant Observability Platform

One-line summary

Build a lightweight, multi-tenant observability backend that ingests logs, metrics, and traces from many client services, processes them asynchronously, stores them efficiently for analytics, and serves low-latency queries for dashboards and investigations.

This is the project to build if the goal is to demonstrate backend depth, distributed systems thinking, and system design maturity for product-company interviews.

Why this project is strong

This project is valuable because it naturally showcases the system design concepts interviewers care about most:

- event-driven architecture
- high-throughput ingestion
- multi-tenancy and tenant isolation
- partitioning and sharding
- caching and Redis key design
- idempotency and deduplication
- retries and dead-letter queues
- backpressure handling
- storage design for time-series data
- consistency trade-offs
- rate limiting
- observability of the system itself
- horizontal scaling
- schema versioning and evolution

It also gives you a strong story in interviews because it is close to real infra work, not a generic CRUD app.

Final project idea

Project name

PulseLens

Project pitch

A multi-tenant observability platform that accepts telemetry from multiple services, assigns each event to the correct tenant, streams it through Kafka, processes it with asynchronous workers, stores it in a time-series optimized store, and exposes a query layer and dashboard for logs, metrics, and traces.

What makes it different from common projects

Most common backend projects are:

- URL shorteners
- e-commerce clones
- chat apps
- Uber clones
- notification systems

Those are fine, but they are too common or too shallow unless implemented with strong infra concepts.

This project is better because it combines:

- real infrastructure patterns used by backend teams
 - multi-tenancy, which is a strong signal for SaaS and enterprise systems
 - analytics + ingestion + query paths
 - operational concerns like retries, DLQs, quotas, and monitoring
-

Problem statement

Modern backend systems produce huge volumes of telemetry:

- request logs
- service metrics
- tracing spans
- error events
- audit events

Teams need a way to:

- ingest this data from many services
- keep tenants isolated
- process events reliably
- query recent and historical telemetry quickly
- detect failures and anomalies
- scale without data loss

The project solves that problem by building a simplified version of an observability backend.

Scope

This should be medium-to-large in scope, not tiny and not endless.

In scope

- multi-tenant ingestion API
- auth with tenant context

- event validation and schema versioning
- Kafka-based event stream
- worker consumers for logs, metrics, and traces
- retry mechanism
- dead-letter queue
- idempotency handling
- rate limiting per tenant
- Redis caching and metadata storage
- time-series storage in ClickHouse or PostgreSQL with partitions
- query APIs for dashboards
- basic dashboard UI or Grafana-based visualization
- system metrics and self-observability
- Docker-based local deployment
- load testing setup

Out of scope

- full enterprise-grade authentication platform
 - complex billing system
 - perfect distributed consensus
 - full OpenTelemetry compliance from day one
 - full visual polish
 - production cloud deployment with paid infra
-

Target users

Primary users

- backend engineers
- SRE / DevOps teams
- platform teams
- developers running multiple services

Secondary users

- small teams building internal tooling
 - startups needing a lightweight monitoring layer
-

Core features

1. Multi-tenant ingestion

Each request belongs to a tenant. The system must ensure that telemetry from one tenant never leaks into another tenant's query results.

2. Event types

Support at least these event types:

- logs
- metrics
- traces
- custom events

3. Async processing

Use Kafka to decouple ingestion from storage.

4. Storage optimization

Use a store that handles large time-series data efficiently.

5. Query API

Support filtering by:

- tenant
- service name
- event type
- time range
- severity
- trace ID / request ID

6. Rate limiting

Prevent noisy tenants or abusive clients from overwhelming the system.

7. Idempotency

Avoid duplicate writes when retries happen.

8. DLQ

Send malformed or repeatedly failing events to a dead-letter queue.

9. System observability

Track latency, queue lag, ingestion rate, error rate, and storage performance.

10. Basic dashboard

Provide a minimal interface to inspect telemetry and query results.

High-level architecture

Request flow

1. Client service sends telemetry to ingestion API.
2. API authenticates the tenant.
3. API validates payload and applies rate limiting.
4. API assigns an event ID and publishes event to Kafka.
5. Consumer services process the event asynchronously.
6. Events are deduplicated and transformed.
7. Events are written to storage.
8. Query API reads data from storage and returns filtered results.
9. Dashboard visualizes the results.

Main components

- Ingestion API
 - Auth / tenant resolver
 - Rate limiter
 - Kafka cluster
 - Consumer workers
 - Deduplication layer
 - Storage layer
 - Query API
 - Dashboard
 - Monitoring stack
-

Architecture diagram text

Use this mental picture for the design:

Client services -> Ingestion API -> Kafka -> consumers -> storage -> Query API -> dashboard

A tenant resolver must attach tenant context at the very first edge and preserve it throughout the pipeline.

Detailed modules

1. Ingestion API

Responsibilities:

- receive telemetry payloads
- authenticate client or API key
- resolve tenant identity
- validate schema
- rate limit per tenant and per API key
- generate event ID if absent

- publish to Kafka
- return success quickly

Important design considerations:

- keep it stateless
- never do heavy storage work synchronously
- fail fast on malformed payloads
- support batching for high-volume ingestion

2. Tenant resolver

Responsibilities:

- extract tenant ID from JWT, API key, or request headers
- enforce tenant-scoped access everywhere
- attach tenant context to logs and metrics

This is one of the biggest signals in the project because it shows you understand multi-tenancy, which is common in real SaaS systems.

3. Rate limiter

Responsibilities:

- protect ingestion endpoints
- prevent tenant abuse
- support per-tenant quotas
- optionally support burst limits

Suggested approaches:

- token bucket
- sliding window
- leaky bucket

Redis can be used to store counters and TTL-based limits.

4. Kafka stream

Responsibilities:

- buffer telemetry events
- support horizontal scaling
- allow replay
- decouple ingestion from processing

Design considerations:

- topic per event type or per domain
- partitioning strategy by tenant ID or service ID
- consumer groups for scaling

- replay support for backfills

5. Consumer services

Separate consumers for:

- logs
- metrics
- traces
- alerting or anomaly detection
- aggregation jobs

Design considerations:

- consumers must be idempotent
- handle poison messages safely
- retry transient errors
- send permanent failures to DLQ

6. Deduplication layer

Needed because event pipelines are often at-least-once.

Options:

- unique event ID + write dedupe key in Redis
- storage-level upsert
- idempotent consumer logic

7. Storage layer

Possible options:

- ClickHouse for analytics and time-series querying
- PostgreSQL with partitions for a simpler version

What the storage layer must support:

- fast time-range queries
- tenant-based filtering
- service-based filtering
- low-cost retention policies
- batch inserts

8. Query API

Responsibilities:

- return logs, metrics, traces
- support filters and pagination
- hide cross-tenant data
- support dashboard queries efficiently

9. Dashboard UI

Keep it minimal.

Possible views:

- latest logs table
- metrics charts
- trace search
- tenant selector
- service selector
- time range filter

A simple frontend is enough. The main value is the backend.

10. Monitoring stack

Monitor the platform itself using:

- request rate
- p95/p99 latency
- Kafka lag
- consumer failures
- retry counts
- DLQ size
- Redis hit rate
- storage write latency
- query latency

This is a meta advantage because you are building observability for an observability platform.

Data model

Core entities

Tenant

- tenant_id
- name
- plan
- quota
- created_at

Service

- service_id
- tenant_id
- name
- environment
- tags

Event

- event_id
- tenant_id
- service_id
- event_type
- timestamp
- payload
- severity
- trace_id
- version

Metric record

- tenant_id
- service_id
- metric_name
- metric_value
- timestamp
- labels

Log record

- tenant_id
- service_id
- level
- message
- timestamp
- trace_id
- attributes

Trace span

- tenant_id
- trace_id
- span_id
- parent_span_id
- service_id
- start_time
- end_time
- status
- attributes

DLQ record

- event_id
- tenant_id
- payload
- failure_reason
- retry_count
- timestamp

Key-value design in Redis

Use Redis intentionally, not randomly.

Good key patterns

- `tenant:{tenant_id}:rate_limit`
- `tenant:{tenant_id}:quota`
- `tenant:{tenant_id}:recent_errors`
- `event:{event_id}:processed`
- `trace:{trace_id}:seen`
- `service:{service_id}:hot_metrics`

Why this matters

This shows you understand real key-value store design:

- namespaces
- TTL usage
- collision avoidance
- per-tenant separation
- hot-path caching

What to cache

- tenant metadata
- API key lookup results
- rate limiting counters
- recent query results
- hot dashboard aggregates

Consistent hashing use case

Consistent hashing is optional, but it is a strong bonus if used thoughtfully.

Where it can fit

- route tenants to workers or partitions
- keep tenant affinity stable
- reduce reshuffling when nodes are added or removed
- improve cache locality

Good explanation in interviews

If the system grows, consistent hashing helps preserve data locality and reduce rebalancing costs when consumers or shards change.

Do not force it where it does not belong. Use it only if you can explain the trade-off clearly.

Partitioning and sharding strategy

This is one of the most important system design sections.

Kafka partitioning

Choose a key such as:

- tenant ID
- tenant ID + service ID
- trace ID for trace events

Goal:

- preserve ordering where needed
- distribute load evenly
- keep related events together

Storage partitioning

Partition by:

- time range
- tenant ID
- event type

Why:

- time-series queries are usually time-bounded
- tenant isolation is mandatory
- retention becomes easier

Horizontal scaling

- scale ingestion API instances horizontally
- scale consumers independently
- scale query API separately

Rate limiting design

Rate limiting should be part of the final design.

Why it matters

- protects backend stability
- prevents one tenant from monopolizing resources
- is a realistic enterprise requirement

Suggested design

- token bucket or sliding window
- Redis-backed counters
- per-tenant quotas
- per-API-key quotas
- burst allowance

What to discuss

- fixed window vs sliding window
- eventual consistency vs atomicity
- Lua scripting for atomic updates
- what happens when Redis is down

This is a strong system design talking point because it shows operational maturity.

Reliability design

Idempotency

You must design for retries.

How:

- every event gets a unique ID
- consumers check whether event already processed
- writes are idempotent
- duplicate delivery does not create duplicate records

Retries

Use retries for transient failures.

Best practices:

- exponential backoff
- limited retry count
- jitter
- retry only transient errors

Dead-letter queue

Permanent failures go to DLQ.

Reasons:

- schema mismatch
- malformed payload
- repeated processing failure

- downstream storage unavailable for too long

Backpressure

If Kafka lag grows or storage slows down:

- slow ingestion acceptance
- shed low-priority traffic
- protect core paths
- surface alerts

This is a very strong interview topic because many candidates never think about backpressure.

Consistency model

This project should be designed mostly around eventual consistency.

Why eventual consistency fits

- ingestion is asynchronous
- telemetry is not always required instantly
- throughput matters more than strict synchronous writes

What is eventually consistent

- query results after ingestion
- dashboard aggregates
- alerts from processed events

What should be strongly enforced

- tenant isolation
- auth checks
- rate limits
- idempotency safeguards

Interview trade-off discussion

You can say that the system favors availability and throughput on the write path, while accepting a small delay before data is queryable.

Schema evolution

This is often missed by candidates, so include it.

Why needed

Telemetry formats change over time.

How to handle it

- version every event schema
- keep backward compatible fields
- ignore unknown fields safely
- validate required fields carefully

Interview angle

This shows you understand how real systems survive product evolution.

Security design

Authentication

Use one of:

- JWT
- API keys
- OAuth-style tenant-scoped token

Authorization

Every query and write must verify:

- tenant ownership
- service ownership
- role permissions if applicable

Security basics

- sanitize payloads
- limit request size
- log securely
- avoid leaking secrets in telemetry

Tenant safety

A tenant must never see another tenant's data.

This is non-negotiable.

Performance goals

You should set explicit goals so the project feels real.

Suggested targets

- ingestion p95 below 200 ms

- query p95 below 500 ms for common time ranges
- zero duplicate writes on retry
- near-zero data loss for accepted events
- support multiple tenants concurrently

Load test goals

Simulate:

- multiple tenants
- thousands of events per second
- burst traffic
- consumer failure scenarios

You do not need real traffic. Synthetic load is enough.

Suggested tech stack

Backend

- Go
- Gin or net/http

Streaming

- Kafka

Caching / atomic counters

- Redis

Storage

- ClickHouse for analytics, or PostgreSQL if you want a simpler start

UI

- minimal React frontend or Grafana

Infra

- Docker
- Docker Compose
- optional Kubernetes later

Monitoring

- Prometheus
- Grafana
- structured logging

Load testing

- k6
 - locust
 - simple Go generator
-

Deployment strategy

Best free strategy

Run everything locally with Docker Compose:

- Kafka
- Redis
- ClickHouse
- API services
- workers
- dashboard

Optional public demo

If you want some deployment:

- deploy only the API/UI layer on a free host
- keep heavy infra local

The project is still impressive without paid cloud infra.

What the codebase should contain

Recommended repository layout

- cmd/ingest-api
- cmd/query-api
- cmd/worker-logs
- cmd/worker-metrics
- cmd/worker-traces
- internal/auth
- internal/tenant
- internal/rate_limit
- internal/kafka
- internal/storage
- internal/dedupe
- internal/dlq
- internal/models
- internal/monitoring
- migrations/
- deploy/

- `docker-compose.yml`
- `README.md`

Why this structure helps

It signals a real backend codebase with separation of concerns, not a tutorial repo.

Documentation deliverables

Your README should be detailed and clean.

Must include

- problem statement
- why this problem matters
- architecture overview
- component responsibilities
- data model
- rate-limiting approach
- partitioning approach
- idempotency approach
- retries and DLQ
- consistency trade-offs
- scaling plan
- local setup instructions
- load testing notes
- screenshots or short demo

Very useful extra docs

- architecture diagram
 - request flow diagram
 - sequence diagram for event ingestion
 - data model diagram
 - failure mode analysis
-

Resume value

When complete, this project gives you lines like:

- Built a multi-tenant observability backend in Go using Kafka, Redis, and ClickHouse
- Designed tenant-isolated ingestion, rate limiting, idempotent consumers, retry pipelines, and DLQ handling
- Supported event-driven processing for logs, metrics, and traces with partitioned asynchronous workers
- Implemented time-series storage and query APIs with low-latency filtering by tenant and service
- Load-tested the system to simulate high-throughput ingestion and burst traffic

These are strong backend resume bullets because they show real system design thinking.

Interview talking points

Be ready to explain:

- why Kafka is used instead of synchronous writes
 - how tenant isolation is enforced
 - how duplicates are prevented
 - why Redis is used for counters and hot keys
 - how you would scale consumers
 - why the storage engine is chosen
 - how you handle retries and failures
 - what happens if Kafka or Redis goes down
 - how you would add sharding later
 - what trade-offs you made between throughput and freshness
 - how you would evolve the schema safely
-

What to avoid

Do not make this project weak by doing the following:

- building too much frontend
- making it a simple CRUD app
- adding too many unrelated features
- pretending cloud deployment is the core value
- skipping failure handling
- skipping multi-tenancy
- skipping load testing
- skipping documentation

This project wins by depth, not decoration.

Build order

Phase 1

- define data model
- create tenant/auth layer
- create ingestion API
- add Kafka producer

Phase 2

- create consumer workers
- add storage writes
- add dedupe and DLQ
- add Redis rate limiting

Phase 3

- create query API
- add caching
- add filtering and pagination
- add dashboard

Phase 4

- add monitoring
- add load testing
- measure latency and throughput
- write documentation

Phase 5

- polish architecture docs
- record a demo
- optimize bottlenecks

Final recommendation

This should be your flagship project.

It is strong because it directly demonstrates:

- backend engineering maturity
- distributed systems knowledge
- system design depth
- enterprise multi-tenancy
- scale and reliability thinking
- practical implementation skill

If you build this well, it gives you a serious story for backend interviews and helps you stand out from candidates who only have common toy projects.

Final project title for your resume

PulseLens — Multi-Tenant Observability Platform

Alternate name

PulseLens — Distributed Telemetry Ingestion and Query System

Final one-line interview summary

I built a multi-tenant observability backend that ingests telemetry events asynchronously, enforces tenant isolation and rate limits, handles retries and DLQs, stores time-series data efficiently, and serves low-latency queries through a scalable distributed architecture.